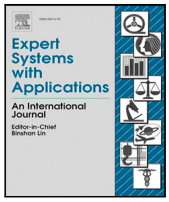
Expert Systems With Applications 254 (2024) 124372

Contents lists available at ScienceDirect

Expert Systems With Applications

journal homepage: www.elsevier.com/locate/eswa



Optimizing fog colony layout and service placement through genetic algorithms and hierarchical clustering

Francisco Talavera, Isaac Lera, Carlos Juiz, Carlos Guerrero *

Computer Science Department, University of Balearic Islands, Crta. Valldemossa km 7.5, Palma, E07121, Spain

ARTICLE INFO

Dataset link: Source code of the genetic algorithm and data of the execution (Original data), Pre-print version (TeX source)

Keywords:
Fog computing
Fog colony
Service placement
Optimization
Hybrid genetic algorithm

ABSTRACT

Fog computing has emerged as a promising paradigm for distributed data processing, but managing numerous devices in fog domains is complex due to the scale of the infrastructure. To address this challenge, organizing fog devices into fog colonies allows independent management on a smaller scale. We present a genetic algorithm (GA) approach that utilizes hierarchical clustering to define the fog colony layout. The GA selects a subset of colony candidates from the dendrogram obtained with hierarchical clustering and optimizes the network communication time between users and applications and the execution time of algorithms that manage application placement in each colony. We deployed an NSGA-II, a multi-objective approach for GAs, to evaluate our proposal. Our experimental results demonstrate that combining a GA with hierarchical clustering improves both optimization objectives. We conducted nine experiment scenarios, varying the number of applications and fog devices. Our results show that even in the worst-case scenario, the GA's results dominated the solutions obtained by two control algorithms after only 137 generations. Additionally, the number of genetic solutions and their homogeneous distribution in the Pareto front were satisfactory.

1. Introduction

Fog computing is a computing paradigm that extends the cloud to the devices located in the network infrastructure (Bonomi, Milito, Zhu, & Addepalli, 2012). These intermediate devices are usually called fog nodes or fog devices, and they own computational resources that allow them to store data or execute applications. Because fog devices are geographically distributed and closer to users, fog computing reduces both the latency between users and applications and the workload of the network infrastructure. This improvement benefits those applications with critical response times. However, fog computing is fundamentally different from cloud computing, due to its higher scale level, wider geographical dispersion of device locations, more limited resources in the devices, and heterogeneity in node interconnection (Guerrero, Lera, & Juiz, 2022; Moysiadis, Sarigiannidis, & Moscholios, 2018). These features result in complexity increment of the infrastructure management. To address this challenge, previous research works have proposed the use of fog colonies, which are a subset of fog devices that act as micro-data centers, and each colony is managed independently (Skarlat, Nardelli, Schulte, Borkowski, & Leitner, 2017). By dividing the fog infrastructure into smaller and independent sets of devices, the complexity of the optimization of the infrastructure is reduced to the size of each colony.

The fog colony layout, which is the organization of fog devices into colonies, directly impacts the overall system performance (Guerrero, Lera, & Juiz, 2018). To the best of our knowledge, very few research works have addressed the optimization of this fog colony layout. To fill this gap, we base the definition of the fog colony layout on the hierarchical clustering of the graph that models the fog devices and their network connections. The hierarchical clustering generates a dendrogram, a tree structure in which its nodes represent a cluster of elements, i.e., a group of fog devices. We consider all the nodes/clusters of the dendrogram as the possible colony candidates, and we select a subset of those clusters, constrained by an exhaustive and disjoint distribution of the devices into colonies, to define the fog colony layout. We solve this decision-making process with an adapted version of the NSGA-II (Deb, Pratap, Agarwal, & Meyarivan, 2002), a multi-objective genetic algorithm (GA).

For each iteration of the GA, a fog service placement algorithm is executed for each colony of each solution in the population of the GA. This hybrid algorithm that combines a GA and a greedy placement algorithm in the subsets (colonies) of the global problem is defined to study the trade-off between the computational complexity of the placement problem and the quality of the solutions. We consider the execution time of the placement algorithm and the response time of

* Corresponding author.
E-mail addresses: f.talavera@uib.es (F. Talavera), isaac.lera@uib.es (I. Lera), cjuiz@uib.es (C. Juiz), carlos.guerrero@uib.es (C. Guerrero).

https://doi.org/10.1016/j.eswa.2024.124372
Received 27 March 2023; Received in revised form 24 May 2024; Accepted 29 May 2024
Available online 4 June 2024

the services deployed in the infrastructure, respectively, as indicators of the computational complexity and the quality of the solutions.

Consequently, the adapted version of our NSGA-II selects the subset of colony candidates that optimizes two objectives: the network communication time between users and applications, and the execution time of the algorithms that manage the placement of the applications in each colony. The use of a multi-objective approach enables the algorithm to find a set of solutions that balance the two objectives. Thus, this paper presents several contributions to the field of fog computing, including::

- The formal definition of the fog colony layout problem and its impact on system performance.
- The use of hierarchical clustering for the definition of the solution space of the genetic-based optimization.
- The development of crossover and mutation operators for a GA that uses the hierarchical-based solutions for the fog colony layout problem.
- The optimization of the network communication time and the execution time of the application placement management algorithm through the designed GA.

By addressing the optimization of the fog colony layout problem, this paper provides a novel approach to improving the performance of fog computing systems. These contributions highlight the potential of using genetic algorithms in fog computing and pave the way for further research in this area.

We evaluate nine experimental scenarios, varying the number of applications and fog nodes. The analysis of these experiments compared the Pareto front of the GA with the solutions obtained from two control algorithms, which demonstrated the superiority of the genetic solutions.

In summary, we hypothesize that the concurrent optimization of the layout of fog colonies and the placement of services overtakes the quality of the results of the scenarios: (a) optimizing service placement globally without considering colonies, and (b) optimizing a predefined colony layout beforehand, with subsequent service placement in this unalterable layout. This hypothesis is validated if there is at least some meta-heuristic that achieves a better improvement in this concurrent optimization. We have selected and studied a hybrid genetic algorithm, which integrates a sub-task focusing on service placement.

The paper is organized as follows: Section 2 reviews related research that has considered using fog colonies. We provide details on the architecture we have defined to enable a dynamic organization of devices into fog colonies in Section 3. Section 4 formally formulates the problem domain and the optimization we are dealing with. We detail the design decisions for implementing the hybrid GA in Section 5. Section 6 presents the experiments and analyzes the results. Finally, Section 7 summarizes the conclusions and outlines future research directions.

2. Related work

The term *fog colony* was first introduced by Skarlat et al. (2017), who defined a framework that supports a hierarchy of fog colonies within a head element, the colony coordinator, in charge of the management of the colony. Since then, many researchers have utilized this conceptual framework in their studies. Several research efforts have focused on adapting existing technologies to the fog landscape (Ogundoyin & Kamil, 2022; Skarlat, Karagiannis, Rausch, Bachmann, & Schulte, 2018), while others have implemented the framework in real-world scenarios (Minh, Huu, Tsuchiya, & Toulouse, 2019).

Optimization processes can be applied to nearly all components of a system. In our case, we can apply optimizations in two phases of the architecture deployment: defining the fog colonies and placing services within the already defined colonies. Furthermore, different solutions can be combined in both phases to achieve optimal results.

A review of the literature indicates that most optimization proposals for fog computing focus on the second phase of the problem, which involves mapping application or service instances to fog colonies and devices. While many optimization algorithms have been developed for this purpose, none of them has addressed the optimization of fog colony layout. Instead, existing solutions are based on adapting traditional optimization algorithms to the fog domain. The largest group of research corresponds to greedy algorithms, mostly based on a first-fit algorithm that allocates services by ordering the nodes prioritizing the utilization of edge devices (Skarlat & Schulte, 2021), the delay-sensitive services (Azizi & Khosroabadi, 2019; Azizi, Khosroabadi, & Shojafar, 2019), the devices closer to the required data (Nguyen & Truong Pham, 2018), or the best value of a weighted function for device resources (Rakshith, Rahul, Sanjay, Natesha, & Ram Mohana Reddy, 2018). A smaller number of proposals are based on optimization algorithms such as search algorithms (Nicopolitidis, Pham-Nguyen, & Tran-Minh, 2019; Rakshith et al., 2018), integer linear programming (Mann, 2022), analytical resolution (Minh, Nguyen, Van Le, Nguyen, & Truong, 2017; Tran, Nguyen, Le, Nguyen, & Pham, 2019), benchmarking (Venticinque & Amato, 2019), multi-dimensional optimization (Nashaat, Ahmed, & Rizk, 2020), self-adapting functions (Ghaferi, Malekhosseini, Rad, & Bagherifard, 2022), genetic algorithms (Skarlat & Schulte, 2021), evolutionary optimization (Ayoubi, Ramezanpour, & Khorsand, 2021; Eslami & Sakhaei-nia, 2021), cuckoo search algorithms (Liu, Wang, Zhou, & Rezaeipanah, 2022), particle swarm optimization (Salimian, Ghobaei-Arani, & Shahidinejad, 2022), imperialist competitive algorithms (Ghobaei-Arani, Asadianfam, & Abolfathi, 2022), gray wolf optimization (Salimian, Ghobaei-Arani, & Shahidinejad, 2021), and fuzzy logic (Tavousi, Azizi, & Ghaderzadeh, 2022). However, none of these solutions address the optimization of fog colony layout, which is the focus of our proposed method.

Our analysis of the experimental phases of these studies suggests that the first phase of defining the fog colony layout is usually performed manually by the infrastructure administrator (Tran, Nguyen, Tsuchiya, & Toulouse, 2020). This distribution is based on the geographical distribution of the fog devices or the size of the fog colonies. In contrast to these proposals, our proposal focuses on the definition of the colony layout, the first phase of the deployment process. Our proposal can be integrated into the first phase of those studies to enhance their optimization performance.

The optimized organization of the fog colony layout, the first phase of the fog infrastructure deployment, has received limited attention in current studies, despite evidence that it directly impacts system performance (Guerrero et al., 2018; Nikolopoulos, Nikolaidou, Voreakou, & Anagnostopoulos, 2022b). This gap in the literature presents a clear opportunity to improve fog infrastructure performance by prioritizing the optimization of the fog colony layout in the initial phase of deployment.

In terms of autonomous organization, Nikolopoulos, Nikolaidou, Voreakou, and Anagnostopoulos (2022a) proposed a framework for self-configured, self-adapted, and context-aware organization of fog nodes into fog colonies. They utilized a publish–subscribe protocol to enable fog nodes to be actively aware of their operating context, share contextual information, and exchange operational policies. Although they explored different communication methods between fog nodes, they did not address the optimization of fog colony layout and its impact on system performance. Similarly, Lordan, Lezzi, and Badia (2021) defined a framework that allows fog nodes to establish relationships with other nodes to create new colonies or participate in existing ones through the use of an API. However, this work did not aim to establish a fog colony organization to optimize the system.

In terms of optimizing the fog layout, we presented a preliminary study (Guerrero et al., 2018) that uses the measures of centrality of the fog infrastructure. First, the fog coordinators are selected using the nodes with the highest centrality values. Once the coordinator candidates are selected, each fog device is attached to the closest fog

coordinator to create the fog colonies. While this process is semi-automatic, the intervention of the infrastructure administrator is still required to determine the number of fog colonies for the selection of fog coordinators.

Tran et al. (2020) highlighted the importance of scalability in the design of fog infrastructures and proposed a model that allows for the automatic addition of new fog nodes into existing fog colonies. Their approach involves the newly added fog node detecting its geographical position and identifying the most suitable colony to join. Similarly, Hatti and Sutagundar (2021) proposed an automatic fog colony determination method based on the clustering of fog devices. In their approach, fog devices in a geographical area are grouped into colonies using a k-means algorithm (Suarez & Salcedo, 2017) that initializes centroids of clusters with the nearest devices based on Euclidean distance.

Our proposal differs from related research by hybridizing genetic optimization and graph partitioning techniques for a non-human-assisted definition of the fog colony layout, aimed at optimizing system performance by finding a trade-off between reducing service placement time and network times. While complex network features are a common solution for optimizing both general distributed systems and fog computing — for example, based on the Girvan–Newman community detection algorithm and the transitive closure (Lera, Guerrero, & Juiz, 2018), a uniformly random community formation (Azimzadeh, Rezaee, Jassbi, & Esnaashari, 2022), or the Kernighan–Lin partition algorithm (Guerrero, Lera, & Juiz, 2020) — , our work is the first to apply hierarchical clustering to design the solution space of the optimization problem and determine the organization of the fog colony layout. Finally, note that there are important surveys that present the current state of the art for the use of GA in the optimization of fog domains (Guerrero et al., 2022). Thus, it is not worthy to include here an extensive analysis of the use of those algorithms, since none of them addresses the optimization of the fog colony layout.

3. Architecture definition

Fog computing can be understood as an extension of cloud architecture, creating a continuum between the cloud and the end-users (Bonomi et al., 2012). In-network devices, located between the data center and the end-users, include computational and storage resources, allowing them to store/process data, execute applications, and more. These intermediate elements of the infrastructure are known as fog nodes or fog devices. Fog devices are geographically distributed and closer to the end-users, reducing latency between them and the nodes where the applications are executed, or the data is stored.

The definition of the fog colony-based architecture is inspired by the work of Skarlat et al. (2017) and it enables a conceptual organization of the computing resources into a layered distribution of responsibilities, both for the execution of applications and the management of the infrastructure.

As it is represented on the left side of Fig. 1, our framework divides the fog devices into groups, called fog colonies, which consist of a single fog coordinator and a set of additional fog devices. The fog coordinator is a fog device that activates the coordination module as needed. Each fog colony has only one fog coordinator, and each fog device belongs to one and only one fog colony.

Fog devices and fog coordinators are capable of deploying and allocating applications. Consequently, there is an implicit organization into layers or levels, with the cloud at the top level, the fog coordinator in between, and the fog devices in the lower level. Each of those layers is assigned with a set of tasks or roles, both from application execution and infrastructure management points of view.

We adopt a multiservice pattern for deploying applications in our framework, which is becoming increasingly popular in fog computing. Applications are composed of multiple services that work together to achieve the application's functionality (Brogi, Forti, Guerrero, & Lera, 2020; Wang & Varghese, 2022). Services can be easily scaled up by downloading and executing their encapsulating elements, or scaled down by stopping and removing instances of the service.

Based on this application execution pattern, the middle section of Fig. 1 shows the distribution of roles among the cloud, fog coordinator, and fog devices. The cloud serves as a central repository for all services and also executes instances of all applications in the infrastructure. Therefore, if an application is not deployed on any fog devices, user requests will be served by the cloud. When a fog device is designated to deploy a service, it downloads the service's encapsulating element (which includes the service's source code and data) from the cloud and starts its execution. The fog coordinator behaves similarly to fog devices when it comes to application execution.

In the case of infrastructure management, right side of Fig. 1, the cloud is responsible for defining the fog colony layout. Once the layout is determined and the fog coordinators are selected, they are notified. Any fog device can act as a fog coordinator when required, enabling self-management of the fog colony layout and dynamic adaptation to changing system conditions. The cloud periodically evaluates the colony layout and decides if changes are necessary.

The tasks of the fog coordinator are to manage the colony, for example by optimizing the placement of the applications inside the colony, and to be in charge of the routing of the communications between colonies. The fog coordinator is aware of the fog devices included in its own colony, and which the coordinators of the other colonies are. The details of the communication between fog devices are explained in the following section (Section 3.1).

The placement of the services for an application is not limited by the fog colony layout. The services for an application can be deployed on the same fog device, on devices in the same fog colony, or on devices in different colonies. The optimization algorithm executed by the fog coordinator will map service instances and devices within its own colony. Users requesting services that are not deployed in the same colony will request them from other colonies or from the cloud.

3.1. Communication patterns

Considering the architecture defined by Skarlat et al. (2017), communication routing between devices is established based on the assumption that a fog device can only communicate with the fog devices in its own colony. This means that the communication time for services within a colony is the sum of the latencies of the shortest path between the devices that host the services.

However, communication between fog devices in different colonies is also possible. In this case, the communication path will go through the fog coordinator of each colony, which acts as a gateway to the other colonies. The communication time for services that are deployed in different colonies is the sum of the latencies of the shortest paths between the fog coordinators of the source and destination colonies, plus the communication times between the fog coordinators and the fog devices in its colony. This communication pattern allows for efficient communication among fog devices while leveraging the benefits of the fog colonies' organization. Fig. 2 shows examples of inter-colony communication with services placed in different colonies, and intra-colony communication with all the services placed in the same colony.

The routing of user requests follows the same assumption as the routing of requests between services, where communication is assumed to be restricted to fog devices within the same colony. If the user and the service are located in the same colony, their requests are routed through the shortest path between the device where the user is connected and the devices that host the service. Whereas, if the service is placed in a different colony, the user request is routed via the fog coordinators.

It might seem that having communication routed through the fog coordinator could make it a single point of failure for communication

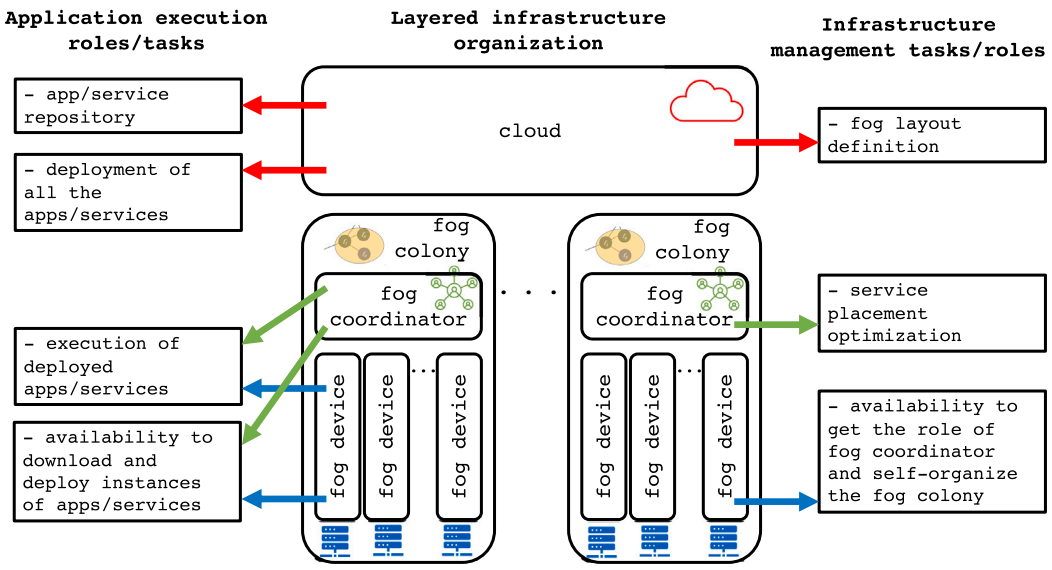


Fig. 1. Layered definition of the infrastructure. The elements in the infrastructure perform both application execution and infrastructure management tasks. Infrastructure tasks: fog layout definition (cloud); service placement (coordinators); latent coordinators (all the fog devices). Execution tasks: repository and deployment (cloud); download, deploy, and execute tasks (fog coordinators and fog devices).

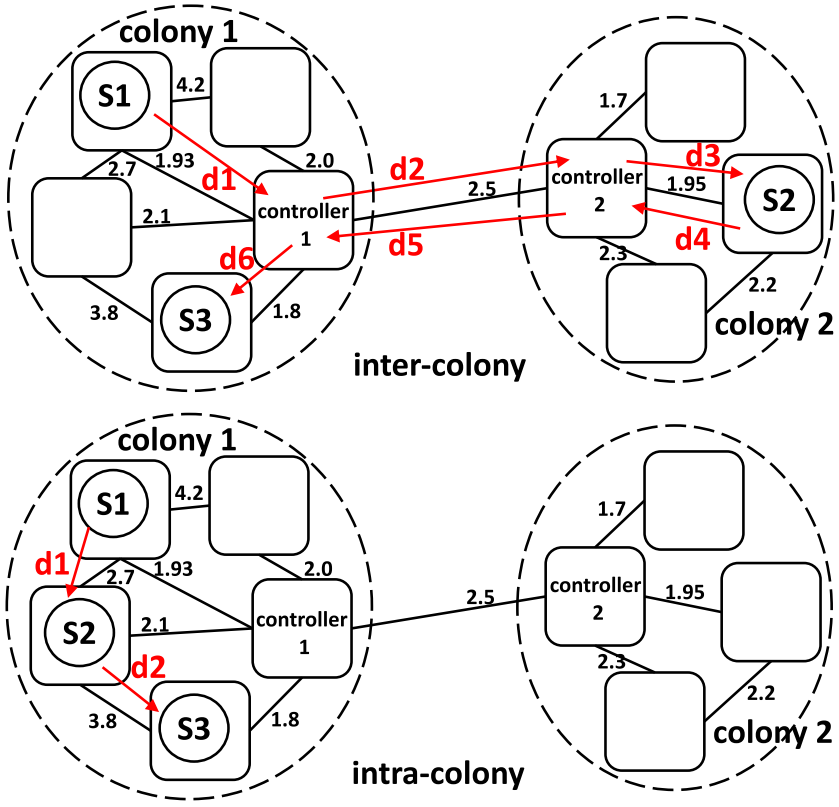


Fig. 2. Examples of communication times concerning the service location for the case of devices in the same colony (intra-colony) and different colonies (inter-colony). Inter-colony: Communication time for an application with services placed in neighbor colonies. The communication takes place through the coordinators of the colonies, with the shortest paths between devices and their coordinators and between coordinators. The first and last services ($S1$ and $S3$) are placed in the same colony, and the second service $S2$ in a neighboring colony. Two inter-colony communications ($d2$ and $d5$) and four intra-colony communications are required, with a communication delay of $d1 + d2 + d3 + d4 + d5 + d6$, that is $1.93 + 2.5 + 1.95 + 1.95 + 2.5 + 1.8 = 12.63$ time units. Intra-colony: Communication time for an application with services placed in only one colony which is reduced to the shortest path between the devices inside the colony. The example only requires two intra-colony communications, $d1 + d2$, equal to $2.7 + 3.8 = 6.5$ time units, with an improvement of almost the double of the previous case.

between devices. But as we explained in the previous section, the fog coordinators are fog devices that activate the coordination module when necessary and, in case of failure, another fog device from the same colony can be selected as the new fog coordinator, ensuring the continuity of the communication. This potential risk can be mitigated by assigning a secondary coordinator who would activate the coordination module in case of the main coordinator's failure. The secondary coordinators should be selected with the same assumptions as the main

coordination, in our case, the centrality metric of the fog devices inside the colony. The number of secondary coordination is limited by the number of devices in the colony, and an ordered list of the devices, using the centrality value, could be used to select the devices for successive failures.

4. Problem formulation

The formal definition of the problem comprises fog devices, interconnection networks, fog colonies, applications composed of services, and users.

Following this order, a fog device, denoted by $f_n \in F$, is a device of the infrastructure that can execute instances of a service. Fog devices are characterized by their resource capacity, $f_n[rc]$. To represent the resource capacity of devices, we use a scalar value rc, without any loss of generality. However, this scalar value could be extended to an n-tuple to model the capacity of specific resources in the device, such as processor, memory, disk, etc.

The fog nodes are connected via a network, with a network link $n_{f_n,f_m} \in N$ representing a direct connection between fog nodes f_n and f_m. Network links are characterized by their latency for transmitting data, denoted by $n_{f_n,f_m}[lat]$.

The fog infrastructure I is defined as the set of fog devices F interconnected by the network links N, and can be modeled as the graph $I = (F, N)$, with fog devices as nodes and network links as edges.

As explained in Section 3, the fog devices are organized into fog colonies, which are disjoint subgraphs of the fog infrastructure I. To obtain the fog colonies, we utilize the dendrogram of the fog devices, obtained with hierarchical clustering.

A dendrogram is a hierarchical binary tree commonly used for organizing elements into clusters (Murtagh & Contreras, 2012). Hierarchical clustering can be performed using an agglomerative (bottom-up) approach, in which each element is considered a cluster at the start, and clusters are iteratively merged until only one cluster remains. Alternatively, a divisive (top-down) approach can be used, where the algorithm starts with all elements in one cluster and iteratively splits it until all clusters contain only one element. The dendrogram reflects the sequential evolution of the algorithm, with each split or join of two clusters represented at a specific height. Fig. 3 depicts an example of a fog infrastructure modeled as a graph and the resulting dendrogram obtained from applying hierarchical clustering.

The dendrogram represents all the candidate fog colonies through its nodes. We identify each candidate fog colony as $C_i \in \mathbb{C}$, and thus the corresponding dendrogram's node is also labeled as C_i. Each C_i corresponds to a subset of fog devices this cluster includes, $C_i \subset F$. A specific fog colony layout, $L_x \in \mathbb{L}$, is defined as a subset of the dendrogram's nodes or candidate fog colonies, $L_x \subset \mathbb{C}$. However, not all possible subsets are suitable colony layouts, as they are constrained by the need for an exhaustive and disjoint distribution of devices between the colonies. A proper colony layout includes each fog device in one and only one cluster ($\exists! C_i \in L_x : f_n \in C_i$), meaning fog devices cannot be included in more than one cluster ($\bigcap_{C_i \in L_x} C_i = \infty$), and each device must be included in a fog colony ($\bigcup_{C_i \in L_x} C_i = F$). The set of proper colony layouts forms the colony solution space $\mathbb{L}$.

Additionally, each fog colony is characterized by the colony coordinator, $f^{C_i}_{coord} \in C_i$, which is the fog device in the colony in charge of managing the colony (Section 3). The colony coordinator is the one with the highest centrality index in the colony, in order to reduce communication latencies between coordinators and between coordinators and colony devices (Lera et al., 2018).

Fog applications ($A_o \in \mathbb{A}$) are comprised of sets of services ($s_p \in S$) that are interconnected through a request/data relationship ($r_q \in R$, where $r = (s_p, s_{p'})$) and modeled as a graph $A_o = (S, R)$. Each service is characterized by its resource requirements ($s_p[rr]$), which are represented using scalar values rr as specified in the definition of fog devices.

These applications are requested by users and things[1] ($u_j \in U$). These users/things are defined as $u_j[A_o, f_n, \lambda]$, where A_o is the application that the user requests, f_n is the fog device to which the user is connected, and λ is the rate at which the application is requested. For the sake of simplicity, we assume that users only request one application. However, if users generate multiple requests, this can be straightforwardly accommodated in our model by defining an additional user for each different service request from the real user. From the perspective of performance evaluation and resource optimization, both modeling approaches are equivalent.

Each service $s_p \in S$ is assigned to a fog device $f_n \in F$ in order to execute the application and respond to user requests. The same service can be assigned to multiple fog devices, and these instances of the service are referred to as s^q_p. The assignment of services to devices is represented by a binary matrix P, where the rows represent the services and the columns represent the fog devices. An element $p_{s_p,f_n} = 1$ if an instance of service s_p is assigned to device f_n, and $p_{s_p,f_n} = 0$ otherwise. Since the placement problem is not addressed locally for each colony placement matrices are defined for each colony, $P^{C_i}\ \forall C_i \in L_x$, instead of a global one. These placement matrices are smaller than the global placement matrix, and as a result, the scale of the placement optimization is reduced, making it less computationally expensive. This is one important reason for the use of fog colonies in the definition of the infrastructure.

The placement is constrained by the resource capacity of the fog devices. The sum of the resource requirements of all the services assigned to a device must be less than or equal to the resource capacity of the device, i.e., $(\sum_{s_p \in S} p_{s_p,f_n} \times s_p[rr]) \leq f_n[rc]$, where s_p is a service p_{s_p,f_n} is the binary decision variable of a service allocated in a device, $s_p[rr]$ is the resource consumption of a service, and $f_n[rc]$ is the available resources in a device.

In summary, this paper addresses the problem of finding the colony layout (L_x) and the placement of the services (P^{C_i}) for each colony (C_i):

$$L_x, \tag{1}$$

$$P^{C_i}\ \forall C_i \in L_x \tag{2}$$

where the colony layout (Eq. (1)) is constrained by a disjoint and exhaustive distribution of the devices into colonies (Skarlat et al., 2017), i.e. all the devices are assigned to at least one colony (Eq. (3)) and no device is assigned to more than one colony (Eq. (4)):

$$\bigcup_{C_i \in L_x} C_i = F, \tag{3}$$

$$\bigcap_{C_i \in L_x} C_i = \emptyset \tag{4}$$

and the service placement (Eq. (2)) is constrained by the resources available in the devices:

$$\left(\sum_{s_p \in S} p_{s_p,f_n} \times s_p[rr] \right) \leq f_n[rc], \forall f_n \in F. \tag{5}$$

where each variable is detailed in the previous paragraph.

4.1. Optimization definition

Our objective is to find the definitions of Eqs. (1) and (2) that optimize the overall execution time for the algorithm that places the services in the devices in a colony (*placement_time*) and the overall response time of the applications (*response_time*).

[1] The term *things* refers to entities or physical objects (end devices) that are capable of requesting the execution of an application to obtain a service or to process the data they generate.

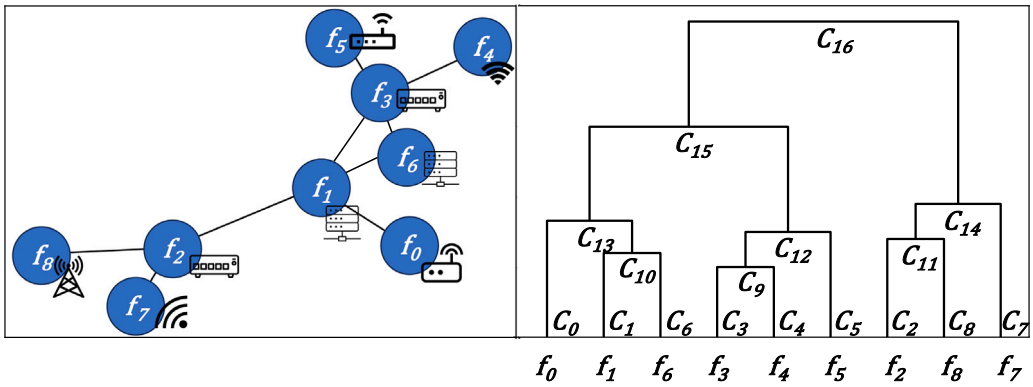


Fig. 3. Example of fog infrastructure (on the left) with nine fog devices ($F = f_0..f_8$) and its dendrogram (on the right) with 17 candidates for fog colonies ($\mathbb{C} = C_0..C_{16}$). The dendrogram is obtained from the hierarchical clustering of the infrastructure's graph. Each node of the dendogram (C_i) corresponds to a colony candidate. Dendogram's leaf nodes (from C_0 to C_7) are the colonies candidates that only include one node. The root node (C_{16}) is the colony candidate that includes all the nodes. Intermediate nodes of the dendogram are cases for colonies with a sub-set of nodes. An example of a suitable layout is $L_{ex} = C_{13}, C_9, C_5, C_{14}$, resulting in the formation of the fog colonies $C_{13} = f_0, f_1, f_6$, $C_9 = f_3, f_4$, $C_5 = f_5$, and $C_{14} = f_2, f_8, f_7$. $L_{ex''} = C_{15}, C_{10}, C_{11}$ is an example of a non-suitable layout since the resulting fog colonies $C_{15} = f_0, f_1, f_6, f_3, f_4, f_5$, $C_{10} = f_1, f_6$, and $C_{11} = f_2, f_8$ violates the disjoint constraint (f_1 and f_6 are included in two fog colonies) and the exhaustive constraint (f_7 is not included in any colony). The agglomeration solution is $L_{agglomerative} = C_{15}, C_{14}$ and the divisive solution is $L_{divisive} = C_0, C_{10}, C_{12}, C_{11}, C_7$.

The optimization of fog service placement is a challenging problem, as it is NP-hard and directly impacted by the number of application instances and fog devices involved in the placement process (Guerrero et al., 2022). As each coordinator independently performs the deployment of services, this process can be executed in parallel in the different colonies. We use the average of the execution time of the placement algorithm in each of the colonies as the indicator of the *placement_time*, formally defined as (Hu et al., 2020):

$$placement\_time = \frac{\sum_{C_i \in L_x} placement\_time_{C_i}}{|L_x|} \tag{6}$$

where $placement\_time_{C_i}$ is the specific execution time of the placement for the colony C_i, L_x is the colony layout, i.e., the subset of selected colony candidates, and $|L_x|$ is the number of colonies in the layout. Due to the complexity of determining this time analytically, we propose to measure it as the real execution time of the placement. This is a common solution when emulation or simulation is required to evaluate the fitness function of an optimization process (Hu et al., 2020). Anyhow, the optimization complexity remains unaltered as the placement of services is already required to evaluate the second optimization objective, the overall response time of the applications (*response_time*).

The overall response time for all the users in the infrastructure, *response_time*, is defined as the mean value of the single response times for all the users (Guerrero et al., 2022):

$$response\_time = \frac{\sum_{u_j \in U} response\_time_{A_o,u_j}}{|U|} \tag{7}$$

where u_j is a user, $|U|$ is the number of users, and $response\_time_{A_o,u_j}$ is the response time of a user requesting an application.

To avoid the influence of fog device resource heterogeneity, we only consider the network communication times for the calculation of the response time, assuming that the execution time of a given service is the same in all the devices, i.e., it is not influenced by the device where the service is executed. This assumption is justified because our optimization process is focused on the fog colony layout, which affects the network time but not the execution time of the services. However, the model can be easily adapted by including the execution time of the services in the devices and considering it as the second component of the response time.

Considering this previous assumption, $response\_time_{A_o,u_j}$ is calculated as the network path between the user (u_j) and the first service of the application (s_{root}) added to the sum of the single network paths for each pair of related services $(s_p, s_{p'})$ of an application $r_q \in R$, where $r_q = (s_p, s_{p'})$ is an edge in the service graph of the executed application (Guerrero et al., 2022):

$$response\_time_{A_o,u_j} = network\_path_{u_j,s_{root}} + \sum_{r_q \in R_{A_o}} network\_path_{s_p,s_{p'}} \tag{8}$$

As we explained in Section 3.1, the network path between two elements (users/services with services) is separated into three possible cases (Skarlat et al., 2017):

- communication inside the same colony. The network path is calculated as the shortest path between the devices that allocate the services.

  $shortestLat(f_n, f_{n'})$

  where f_n is a device.
- communication between different colonies. The network path is calculated as the sum of the three shortest paths between: the origin device and origin coordinator; the origin coordinator and target coordinator of the closest colony; and, the target coordinator and target device.

  $$shortestLat(f_n, f^{C_i}_{coord}) + shortestLat(f^{C_i}_{coord}, f^{C_{i'}}_{coord}) + shortestLat(f^{C_{i'}}_{coord}, f_{n'})$$

  where f_n is a device and $f^{C_i}_{coord}$ is a colony coordinator.
- communication between a colony and the cloud. This is considered when the requested service is not allocated in any colony or when the cloud is closer to the origin requester than the colony allocating the requested service. The network path is calculated as the sum of the two shortest paths between: the origin device and the origin coordinator; and, the origin coordinator and the cloud.

  $$shortestLat(f_n, f^{C_i}_{coord}) + shortestLat(f^{C_i}_{coord}, Cloud)$$

  where f_n is a device, $f^{C_i}_{coord}$ is a colony coordinator and $Cloud$ is the device representing the cloud provider.

The notation to select the minimum values when several alternatives are possible is included to aggregate the three previous cases, resulting in the formal definition of the response time for a single user

request in (Skarlat et al., 2017):

$$network\_path_{s_p,s_{p'}} = \begin{cases} shortestLat(f_n, f_{n'}) & f_n \in C_i,\ \exists f_{n'} \in C_i \mid p_{s_p,f_n} = p_{s_{p'},f_{n'}} = 1 \\ \min\left(closestColony(s_p, s_{p'}), cloudLat(s_p, s_{p'})\right) & otherwise \end{cases} \quad (9)$$

where

$$closestColony(s_p, s_{p'}) = shortestLat(f_n, f^{C_i}_{coord}) + \\ + \min\left(shortestLat(f^{C_i}_{coord}, f^{C_{i'}}_{coord}) \right. \\ \left. + shortestLat(f^{C_{i'}}_{coord}, f_{n'}), \forall C_{i'} \mid \exists f_{n'} \in C_{i'} \wedge p_{s_{p'},f_{n'}} = 1\right) \quad (10)$$

and

$$cloudLat(s_p, s_{p'}) = shortestLat(f_n, f^{C_i}_{coord}) + \\ + shortestLat(f^{C_i}_{coord}, Cloud) \quad (11)$$

where s_p is a service, f_n is a device, C_i is a colony, $f^{C_i}_{coord}$ is a colony coordinator, p_{s_p,f_n} is the binary decision variable of a service allocated in a device, $Cloud$ is the device representing the cloud provider.

This formulation support the idea that the size of the colonies influences in the response times. Smaller colonies are more likely to require shifting the allocation of services to neighboring colonies because of their fewer resource. This increases the number of cases that require communication between colonies. On the contrary, larger colonies are able to allocate more services, reducing the number of cases that requires inter-colony communication. But the network path is bigger for those cases that require of this inter-colony communication, because the fog coordinator is further away (Guerrero et al., 2018).

In summary, the layout of the colony has an influence on both the *response_time* and the *placement_time*, but these effects are opposed when the colony sizes are increased or decreased. This is a common multi-objective optimization problem that cannot be solved analytically due to its size, and therefore requires a meta-heuristic approach (Guerrero et al., 2022). Consequently, the optimization problem is formulated as:

$$\text{Define} \quad L_x, \quad (12)$$

$$P^{C_i}\ \forall C_i \in L_x \quad (13)$$

$$\text{to minimize} \quad response\_time \quad (14)$$

$$placement\_time \quad (15)$$

5. Collaborative and low-level hybridized algorithm proposal

GAs are meta-heuristic search algorithms that work with a population, which consists of a set of solutions to the problem to be optimized. GAs improve this population through an evolution process based on the stochastic combination of the best solutions (Holland et al., 1992). The solutions of a GA evolve over generations, limited by a finish condition. Each new generation is created by selecting, combining, and modifying solutions from the previous generation, creating an offspring of the same size (number of solutions) as the population size.

We propose hybridizing the GA that determines the fog colony structures, with a first fit decreasing greedy algorithm for service placement inside the colony. Throughout the evolutionary optimization process, along generations, the greedy algorithm is executed for each colony and solution to determine the service placement inside the colonies. This is the point in time at which the execution time of the placement algorithm is gauged. At this stage, the colonies are already established for a given generation of the GA. In the next GA steps, new solutions (colonies structures) will be defined, and new executions of the greedy algorithm will be required.

In general, strategies that combine GAs and other heuristics are classified using the execution flow (relay or collaborative) and the degree of coordination (low-level and high-level) (Rodriguez, Garcia-Martinez, & Lozano, 2012; Talbi, 2002). Our proposal is collaborative and low-level, since the greedy algorithm is incorporated as a subordinated component integrated into the GA and executed for each iteration of the GA, in particular, for the evaluation of the fitness function.

The objective of the hybridized proposal is to evaluate the trade-off between the execution time of the placement algorithm (which slows as the colonies diminish due to smaller problem size) and solution quality (which increases as the colonies are bigger due to a more global view of the placement problem), measured in terms of response time.

5.1. Genetic optimization of fog colony layout

We tackle the optimization of the fog colony layout using the Pareto dominance concept, which is more effective than the weighted sum approach for multi-objective optimization problems (Srinivas & Deb, 1994). This method aims to find a set of optimal solutions in the solution space, called the Pareto set, and its image in the objective space, the Pareto front, which represents the trade-offs between the optimization objectives. This allows the decision-maker to choose the solution that best fits their preferences.

NSGA-II (Elitist Non-Dominated Sorting Genetic Algorithm) (Deb et al., 2002) is a widely used GA for solving multi-objective optimization problems based on the Pareto dominance concept. The selection of NSGA-II is based on is ability to handle nonlinear and non-convex objective functions, find multiple Pareto-optimal solutions, and incorporate domain-specific knowledge. Additionally, NSGA-II performs well on problems with two and three objectives, outperforming many other algorithms in terms of solution quality and convergence speed (Guerrero et al., 2022; Von Lücken, Barán, & Brizuela, 2014; Zitzler, Laumanns, & Thiele, 2001). Additionally, NSGA-II is one of the most popular GA implementations for multi-objective implementations in fog computing resource management, as shown in several recent surveys (Jalali Khalil Abadi & Mansouri, 2024; Manihar, Patel, & Agrawal, 2024). In any case, our proposal does not constraint the use of a specific GA algorithm, and other GAs could also be implemented because our objective is not to study the benefits of the NSGA-II, and we aim to study the improvements of concurrent optimization of colony layout and service placement in front of their isolated optimization.

NSGA-II mainly differs from traditional single objective GAs in how the goodness of the solutions is calculated to be compared between them. NSGA-II is a Pareto-based algorithm that uses the concept of dominance to order the solutions.[2] NSGA-II recursively classifies the solutions into fronts, which are created with the solutions that are not included in previous fronts and that are not dominated by any other solution. Solutions in the former fronts have higher goodness than solutions in the latter fronts. Solutions inside the fronts are ordered by the crowding distance, which is calculated as the minimum Euclidean distance of the objective values from one solution to the others. NSGA-II considers that dispersed solutions are more significant than the solutions that are concentrated together.

The effectiveness of a GA implementation depends on various factors, such as solution representation, crossover and mutation operators, offspring selection, fitness function, selection operator, and parameter calibration (Larranaga, Kuijpers, Murga, Inza, & Dizdarevic, 1999). While NSGA-II already includes some default elements such as elitism for offspring selection and binary tournament for the selection operator, the other factors require careful consideration and customization to improve the optimization results.

Chromosomes are the solution representations in a GA. They are commonly modeled with an array or a matrix. In our particular case, genetic optimization searches for a suitable fog colony layout. As

[2] A solution s_1 non-dominates solution s_2 if s_2 is not better for any objective and s_1 is better for at least one objective.

Algorithm 1 Multi-objective genetic optimization algorithm (Deb et al., 2002)

```
procedure NSGA-II
    P_t ← generateRandomPopulation(pop_size)
    placement, fitness.placement_time ← servicePlacement(P_t)
    fitness.response_time ← calculateNetworkDistance(P_t, placement)
    fronts ← calculateFronts(P_t, fitness)
    distances ← calculateCrowding(P_t, fronts, fitness)
    for i in 1..gen_num do
        P_off = ∅
        for j in 1..pop_size do
            if random(0,1) < ρ_cross then
                father1 ← binaryTournament(P_t, fronts, distances)
                father2 ← binaryTournament(P_t, fronts, distances)
                child1, child2 ← crossover(father1, father2)
            if random(0,1) < ρ_mut then
                rndOp ← random(0,1)
                if rndOp < ρ_mut^join then
                    joinMutation(child1),joinMutation(child2)
                else if ρ_mut^join > rndOp < ρ_mut^split then
                    splitMutation(child1),splitMutation(child2)
            if random(0,1) < ρ_rep^agg then
                repairAgglomerative(child1),repairAgglomerative(child2)
            else
                repairDivisive(child1),repairDivisive(child2)
            P_off = P_off ∪ {child1, child2}
        placement, fitness.placement_time ← servicePlacement(P_off)
        fitness.response_time ← calculateNetworkDistance(P_off, placement)
        P_union = P_off ∪ P_t
        fronts ← calculateFronts(P_union, fitness)
        distances ← calculateCrowding(P_union, fronts, fitness)
        P_union = orderElements(P_union, fronts, distances)
        P_t = P_union[1..pop_size]
    Solution = fronts[1] #the Pareto front
```

we explained in Section 4, the colony layout is defined through a subset of nodes of the dendrogram, particularly, the selected candidate fog colonies. Accordingly, we represent the solutions using a dendrogram with binary labeled nodes that indicate the selected candidate fog colonies. For internal storage and processing, this dendrogram is transformed into a binary array chromosome of size the number of candidate fog colonies ($|\mathbb{C}|$), where each position of the array represents a candidate colony (a node of the dendrogram), and the binary value of the position indicates if the candidate colony is selected (value 1) or not (value 0). Fig. 4 shows an example of our chromosome representation. It is also important to remark that, although the solutions are represented with a binary array, the crossover and the mutation are based on a tree structure.

Algorithm 1 presents the basic structure of our modified version of NSGA-II, which evaluates fitness functions by emulating the placement process of services into colonies. In summary, the algorithm creates an initial random population, and it evolves over generations by selecting pairs of solutions that are combined through the crossover and mutation operators.

As is usual in GAs (Gibbs, Maier, & Dandy, 2015), we performed a preliminary exploratory phase to calibrate the algorithm and define the population size (pop_{size}), the number of generations gen_{num}, and the operators' probabilities (ρ_{cross} for crossover and ρ_{mut} for mutation). We also adapted the operators to our chromosome representation, except for the binary tournament selection, the default selection operator in the NSGA-II (Deb et al., 2002). Each parent is selected as the best solution between two random ones. Similarly, NSGA-II implements elitism (Natesha & Guddeti, 2021) also by default, which preserves the best N solutions from the current generation in the next generation.

We implemented a sub-tree crossover (Buijs, van Dongen, & van der Aalst, 2012), which is similar to the one-point crossover but adapted for a tree-based structure. This crossover randomly selects a node from the dendrogram, and the sub-tree formed by the selected node and all its descendants is exchanged between the two parents. Fig. 5 provides an example of this procedure.

Mutation operators are used in GAs to prevent getting stuck in the local minima of the solution space. We define two mutation operators, colony-join and colony-split. They are randomly selected with probabilities ρ_{mut}^{join} and ρ_{mut}^{split} when a mutation is triggered.

The colony-join mutation selects a candidate colony at random (indicated by a value of 1 in the chromosome) and replaces it with its parent colony. This results in the previously selected colony and its sibling (the other child of its parent) forming a new single colony. Conversely, the colony-split mutation selects a candidate colony at random (also indicated by a value of 1 in the chromosome) and splits it into two separate colonies corresponding to its children. Fig. 6 provides an example of the outcomes for both mutation operators.

New solutions obtained through crossover or mutation may not satisfy the constraints defined in Section 4, resulting in infeasible solutions. To ensure feasibility, we apply a repair operator to adjust the solutions. Alternatively, some GA designs assign an infinite fitness value to infeasible solutions, but we rejected this approach as it would result in many infeasible solutions in the population.

The repair operator checks whether: (a) a selected colony (content) is a subset of a bigger also selected colony (container); (b) all the devices are part of one colony. Two opposite repair operators are defined, the agglomerative one and the divisive one, that are randomly selected to transform incorrect solutions into correct ones, with probabilities ρ_{rep}^{agg} and ρ_{rep}^{div}, respectively. In the case of the agglomerative repair operator, it first keeps the container colony selected and deselects the content one. Non-included devices are grouped into colonies when possible; if not, a one-element colony is created for the device. Both sub-tasks result in reducing the number of colonies. On the contrary, the divisive repair operator keeps the content colony selected and deselects the container one. A colony of one-element size is selected for each fog device that is not included in any colony. Both sub-tasks result in increasing the number of colonies.

5.2. Subordinated service placement algorithm for fitness calculation

The fitness calculation in our proposal consists of two components: the overall service response time (*response_time*) and the placement algorithm execution time (*placement_time*). For each new child in the population, we calculate both measures after executing a placement algorithm as a subordinated sub-task of the GA. We use a first-fit greedy algorithm for service placement (Lera et al., 2018), but this algorithm can be easily replaced by another optimization heuristic (Nayeri, Ghafarian, & Javadi, 2021).

The time taken by each execution of the placement algorithm is measured, obtaining the *placement_time* component for each new individual. Subsequently, we can evaluate the estimated application response time using Eq. (7), obtaining the second component of the fitness, *response_time*.

A first-fit greedy algorithm for service placement orders the services to be placed in decreasing order using a given criterion, and each service is placed into the first device where it fits. If after considering all the devices in the colonies, the service is not placed anywhere, its placement is shifted to other neighboring colonies.

Algorithm 2 shows the baseline of our placement algorithm. The services requested in a colony are ordered by popularity, from the most to the least requested. The devices are assigned by selecting the closest devices to the users where the services fit. If there is no

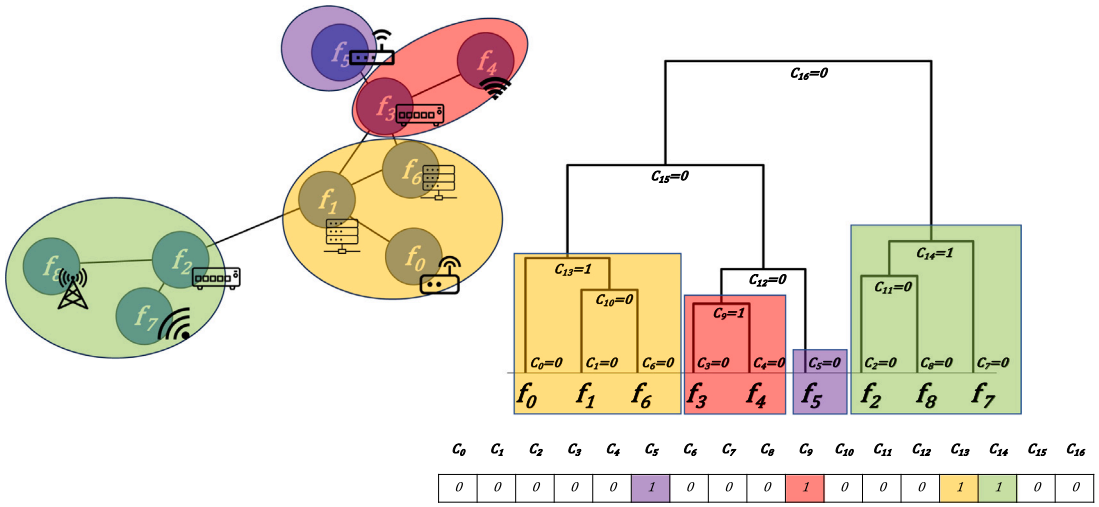


Fig. 4. Example of the chromosome of a possible solution for the fog colony layout in Fig. 3 with four colonies, $L_{ex} = C_{13}, C_9, C_5, C_{14}$. Each chromosome element corresponds to a dendogram's node. Chromosome's positions with value 1 indicate the selected colony candidates. For example, $C_{13} = 1$ indicates that the dendogram's intermediate node C_{13} determines the subset of infrastructure's nodes for the colony (f_0, f_1, and f_6). The selected dendogram's nodes need to satisfy the constraints of that all the fog devices are included in one and only one colony (disjointed and exhaustive distribution of the fog devices).

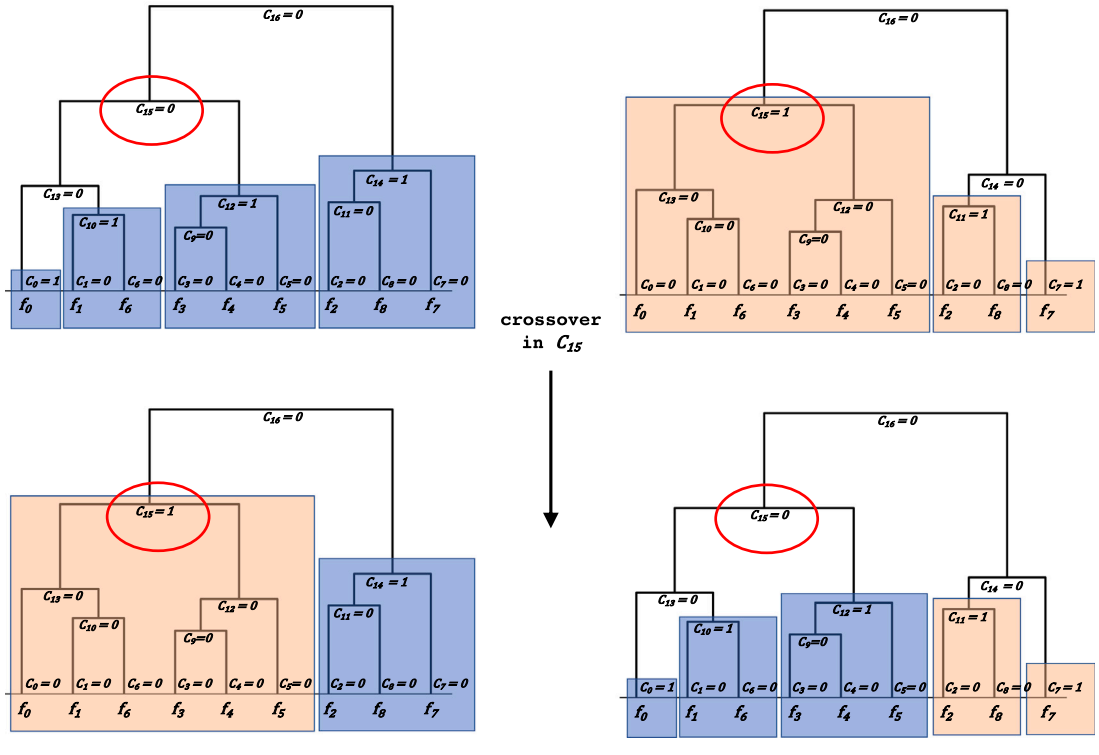


Fig. 5. Example of the crossover operator. The dendogram node C_{15} is selected as the crossing point. The subtrees from each solution are interchanged to create two children. The first child of the example (left side) incorporates the values of the second parent (orange one) for the randomly selected node and the below ones — nodes C_0, C_1, C_3, C_4, C_5, C_6, C_9, C_{10}, C_{12}, C_{13} and C_{15} — and the values of the rest of nodes from the first parent (blue one) — nodes C_2, C_7, C_8, C_{11}, C_{14} and C_{16}. The second child is generated analogously, but by taking the inverse cases.

device where the service fits, the placement of this service is shifted to the neighboring colony. These shifted services are also included in the initial ordered list of services, but priority is given to the services requested from the colony. This iterative and greedy process of placing the services is repeated in parallel in each colony of the layout. Finally, if any service cannot be allocated in the neighbor colonies, it would be executed in the cloud, because the cloud can be provisioned with enough resources to execute all the services in the system.

5.3. Summary of the innovative aspects of the hybridized proposal

Our hybridized proposal of NSGA-II, which integrates greedy placement into the iterative evolution of genetic optimization, offers a more flexible optimization due to the coordinated and simultaneous optimization of colony layout and service placement. A non-integrated version would first optimize the colony layout and, once the final structure of the colonies is defined, the optimal service placement

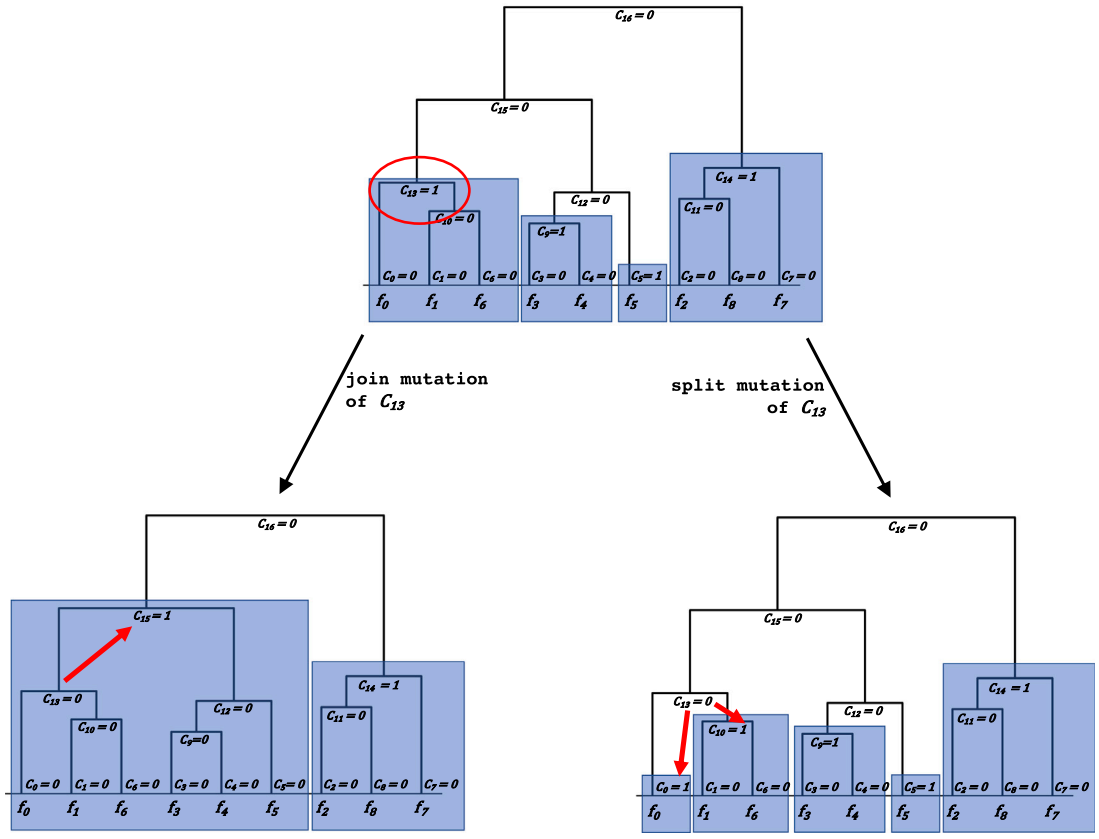


Fig. 6. Example of the join (left) and split (right) colonies mutation on candidate colony C_{13}. For the join mutation example, C_{15} is selected as the new colony and the agglomerative repair operator includes colonies C_9 and C_5 in the newly selected colony C_{15} to keep a disjoint colony layout. The split mutation example divides the colony C_{13} into its two new children colonies, C_0 and C_{10}.

Algorithm 2 First-fit decreasing greedy service placement algorithm

Input
*2 *requestedServices* Services requested by users in the colony.
*2 *colonyDevices* Devices in the colony.
*2 *shiftedServices* Services with placements shifted to this colony

function GREEDYPLACEMENTINCOLONY
  $S_r \leftarrow orderByPopularityDesc$(requestedServices)
  $S_{sh} \leftarrow orderByPopularityDesc$(shiftedServices)
  $S_{ord} \leftarrow S_r + S_{sh}$
  for s in S_{ord} **do**
    $userDevices \leftarrow getDevicesWhereRequested(s)$
    $D \leftarrow orderByMeanDistanceAsc(colonyDevices, userDevices)$
    for $d \in D$ **do**
      if $fit(s,d)$ **then**
        $placement[s][d] = 1$
        break
    if not $fit(s,d)$ **then**
      $c \leftarrow getNeighbouringColony(this)$
      $c.shiftedServices.add(s)$
  return *placement*

would be searched without the possibility of modifying the colonies, which is a less flexible process.

To the best of our knowledge, this is the first proposal to simultaneously optimize the colony layout and the service placement with a collaborative and low-level hybridized optimizer that integrates a greedy algorithm as a subordinated task of a GA. The objective is to find the trade-off between reducing the complexity of high-scale scenarios optimization, such as fog computing infrastructures, without losing the flexibility and the quality of a global, but computationally demanding, optimizer.

Recent optimization studies lack high-demand computational requirements, with very variable conditions because of their high dynamics, and a lack of simultaneously combining different heuristics in the same optimization process (Guerrero et al., 2022). The impact of our proposal is in allowing a new set of types of optimization processes by proposing a first step in reducing the complexity of the optimization, by using fog colonies and combining heuristics, by alternating different optimizers for the colony layout definition and the service placement.

It is important to clarify that the solution quality is only warranted if the optimization algorithm is re-executed to accommodate changes in network topology or failures in coordinators or other fog devices. An approach to avoid the need for re-optimization in, for example, the case of coordinator failures would be integrating indicators of coordinator failure costs as optimization objectives in the GA. This would consider the cost associated with reallocating coordinators when fog colonies definition was performed.

6. Experimental evaluation

The experiments are designed to evaluate the optimization ratio of our proposal, to study the goodness of the results, and to compare our proposal with the other two alternatives. But before addressing

Table 1
Genetic parameters values for the experimental phase.

Parameter		Value
Population size	pop_{size}	100
Number of generations	gen_{num}	–
Crossover probability	ρ_{cross}	0.95
Mutation probability	ρ_{mut}	0.3
Selection probability of mutation operators	ρ_{mut}^{join}, ρ_{mut}^{split}	0.5, 0.5
Selection probability of repair operators	ρ_{rep}^{agg} ρ_{rep}^{div}	0.5, 0.5

Table 2
Scenario configuration values for the experimental phase.

Feature	Value
Infrastructure	
Number of fog devices	{50, 100, 200, 300}
Infrastructure topology	Barabasi–Albert
Coordinator selection	Betweenness centrality
Fog network latency (ms)	[2..6]
Cloud network latency (ms)	100
Fog device resources (resource units)	[1..4]
Percentage of gateways fog devices (%)	25
Application	
Number of applications	{20, 40, 60}
Application resources (resource units)	[1..2]
User	
User inter-request time (ms)	[5..10]
Application popularity per gateway (%)	[0..75]

these issues, we first detail the experiment configuration in terms of the genetic optimization parameters and the experiment scenario outline.

All the parameters defined in Section 5.1 strongly influence the optimization process (Birattari & Kacprzyk, 2009), and they need to be tuned prior to the experiments' execution. The calibration of these parameters is typically performed during a pre-exploratory phase (Mosayebi & Sodhi, 2020), in which a range of values is studied to determine the most suitable parametrization. In this exploratory phase, we studied and fixed the values of the population size, the number of generations, and the probabilities of the different operators (Table 1).

The experimental scenarios are defined by the fog infrastructure, the applications deployed in the system, and the users connected to it. The configuration parameters are summarized in Table 2. We chose the Barabasi–Albert topology for the network infrastructure and the Betweenness centrality index for the coordinator selection because they resulted in the topology and centrality index that most benefit the network times in a fog colony-based layout (Guerrero et al., 2018). The system workload was defined in terms of the number of gateways and the number of users per gateway that request each application (application popularity). The remaining parameters were based on the experiments in Gupta, Vahid Dastjerdi, Ghosh, and Buyya (2017).

We compare the results of our approach with two control algorithms: one with only one colony and the other one considering a fixed colony size (Guerrero et al., 2018). All the experiments are implemented in Python and the source code is available in a public repository.[3] Our proposal cannot be compared with any other state-of-the-art solution because, to the best of our knowledge, there are no other alternatives that optimize the fog colony layout as we explained in the related work section (Section 2).

The first control algorithm, labeled as *one-colony*, does not consider colonies in the fog layout, and it is functionally equivalent to having only one colony that includes all devices. Therefore, a colony optimization process is not necessary for this case. The optimization process is limited to a global optimization of service placement. To ensure a fair comparison of the influence of colony layout optimization, we also use a greedy placement algorithm (Gupta et al., 2017). This control algorithm is equivalent to an algorithm that only optimizes the service placement over the whole infrastructure.

We also compare our proposal with a second control algorithm, labeled as *fixed-size*, that optimizes the fog colony layout by determining the best-fixed size for all the colonies (Guerrero et al., 2018). This size is selected in a previous human-assisted process. In this previous phase, we determined an average colony size of 5 devices. This control algorithm is equivalent to an algorithm that only optimizes the colony layout separately.

6.1. Results

The experimentation includes several fog infrastructure sizes combining the number of fog devices (50, 100, 200, or 300 devices) and the number of applications (20, 30, or 60). This results in the execution of nine experiment scenarios. For each scenario, we execute and compare the results of our GA-based approach, with the two control algorithms: *one-colony* and *fixed-size*.

Due to the stochastic nature of the GA, the GA execution is repeated ten times, and the mean value and the standard deviation of the GA results are calculated and presented. Contrary, the control algorithms are executed once because they are deterministic, and the result of the execution is always the same. Moreover, the NSGA-II returns a Pareto front and the control algorithms a single solution. Consequently, the comparison between the algorithms involves comparing a Pareto front with a single solution.

In the first step of our result analysis, we compare the *fixed-size* and *one-colony* solutions with one solution selected from the Pareto set of the GA. We select the solution considering the smallest Euclidean distance between the normalized values of the solution's objectives and the origin point of the objective space. The Euclidean distance is suitable for the selection of one solution from the Pareto set when both objectives have the same importance (Ferreira, Fonseca, & Gaspar-Cunha, 2007).

The solution selected from the Pareto set is labeled as *smallED-GA*. Fig. 7 shows the Euclidean distance between the three solutions and the origin point of the objective space — point (0,0). The Euclidean distance gives us an idea about which solutions have a better trade-off between both optimization objectives. A smaller distance indicates smaller values of the optimization objectives, i.e., a higher optimization level. Additionally, the distances do not have units because it is calculated with the normalized values of the objectives. The standard deviation values for the 10 executions of the GA range between 0.013 and 0.029, being all of them smaller than a 3%.

Fig. 7 clearly shows that the solution selected from the GA is always better than the other two solutions because of its smaller distance. On the other hand, the *one-colony* algorithm is the worst of the three. However, a more in-depth analysis is necessary due to the nature of the GA, which produces a Pareto set instead of a single solution. In Fig. 8, we present a graphical representation of the objective space of the Pareto front to perform a more detailed analysis.

Fig. 8 helps us to make a more precise comparison of the objective space. Each scatter plot corresponds to one of the nine experiment scenarios, which vary in the number of applications and devices. Each point on the scatter plot represents the values of the two optimization objectives of one solution. In these objective spaces, we represent: orange-colored points as the objective values of the Pareto front obtained with the GA; blue-colored points as the objective values of the solutions obtained with the GA but not included in the Pareto front; a black star as the solution selected from the Pareto front by applying the normalized Euclidean distance; a red-colored cross as the objective values of the solution obtained with the *fixed-size* algorithm; and a green-colored cross as the objective values of the solution obtained with the *one-colony* algorithm.

[3] https://github.com/acsicuib/GA-hierarchical-clustering

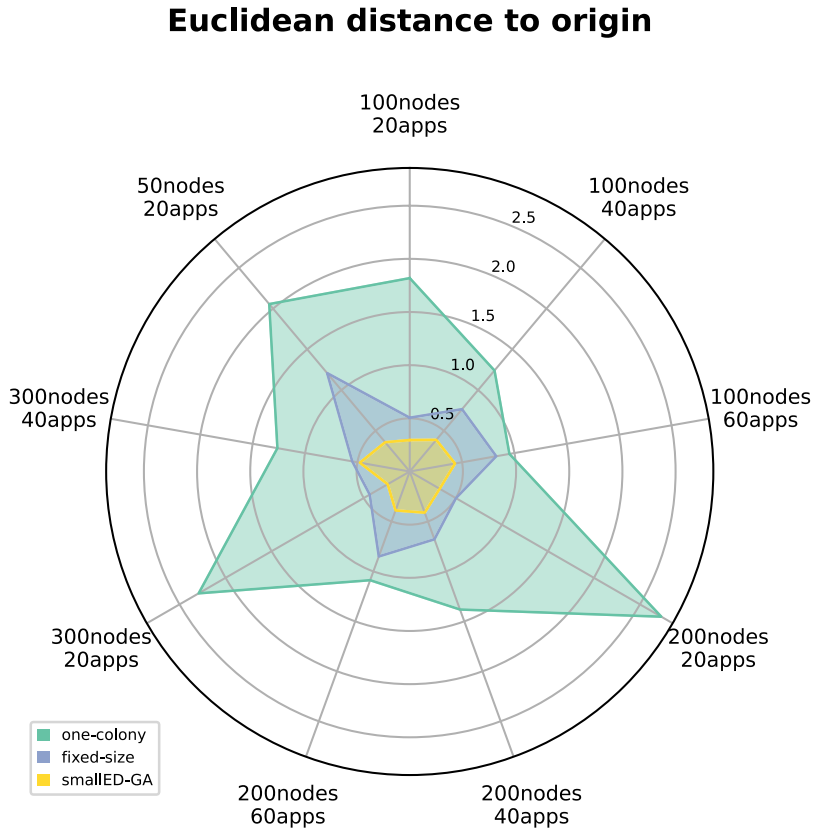


Fig. 7. Comparison of the Euclidean distance to the origin point for the GA selected solution and the other two solutions from the control algorithms.

Table 3
Comparative metrics for the non-dominated solution sets.

Experiment	$S(GA)$	Mean generation when $C(\text{GA, one-colony}) = C(\text{GA, fixed-size}) = 1$	
50nodes 20apps	0.904 ($\sigma = 0.099$)	1.0	($\sigma = 0.0$)
100nodes 20apps	0.627 ($\sigma = 0.135$)	14.4	($\sigma = 4.142$)
200nodes 20apps	0.724 ($\sigma = 0.178$)	75.5	($\sigma = 16.814$)
300nodes 20apps	0.605 ($\sigma = 0.132$)	137.3	($\sigma = 21.391$)
100nodes 40apps	0.726 ($\sigma = 0.071$)	4.4	($\sigma = 0.699$)
200nodes 40apps	0.772 ($\sigma = 0.129$)	14.0	($\sigma = 3.621$)
300nodes 40apps	0.706 ($\sigma = 0.065$)	81.7	($\sigma = 17.098$)
100nodes 60apps	0.873 ($\sigma = 0.100$)	3.4	($\sigma = 0.966$)
200nodes 60apps	0.839 ($\sigma = 0.111$)	7.9	($\sigma = 1.969$)

The scales of the axis have been adapted to the range of values obtained with the GA. Consequently, some scatter plots do not include the crosses that correspond to the control algorithms because they are too far from the Pareto front. For simplicity, we present the scatter plot for the Pareto fronts of only one of the execution repetitions of the GA. However, the results presented in the figure are representative because all the results are homogeneous, and the conclusions can be generalized to all the executions.

The coverage metric is used to compare the Pareto fronts obtained with different optimization algorithms. It is defined as $C(A, B)$ and calculated as the fraction of solutions in B that are dominated by the solutions in A (Zitzler & Thiele, 1999). In our case, where we compare a Pareto front with a single solution, $C(\text{GA, one-colony}) = C(\text{GA, fixed-size}) = 1.0$ indicates that both control solutions are dominated by the Pareto front of the GA. This means that we can find at least one solution in the Pareto front that reduces both optimization objectives in comparison to the control solutions, and that all the other solutions in the Pareto front improve, at least, one of the objectives.

From the analysis of the objective space of the solutions in Fig. 8, we observe that the Pareto fronts of the GA dominate the solutions of the control algorithms in all the experimental scenarios except for the one with 300 nodes and 20 applications. In other words, $C(\text{GA, one-colony}) = C(\text{GA, fixed-size}) = 1.0$ in generation number 100 in all the experiments except for one. This confirms the results observed in Fig. 7, regardless of the selection process of the solution from the Pareto set, and the optimization is better throughout the Pareto set.

Although in the experiment scenario with 300 nodes and 20 applications, the solution obtained from the *fixed-size* algorithm is not dominated by the Pareto front, the control solution is only able to dominate two solutions in the Pareto front, resulting in $C(\text{GA, fixed-size}) = 2.0/28.0$. The other 26 solutions in the Pareto set improve at least one objective of the *fixed-size* solution. To investigate further, we extended the number of generations and observed that the Pareto front dominated the *fixed-size* solution on average from generation number 137. Thus, the lower optimization of the GA in this experiment scenario is explained by an underestimation of the finish parameter.

It is also interesting to perform a similar analysis for all experiment scenarios. Table 3 presents, on average, the number of generations required for the Pareto front to dominate both control solutions, i.e., $C(\text{GA, one-colony}) = C(\text{GA, fixed-size}) = 1.0$. In most cases, the number of generations required to improve the solutions obtained with

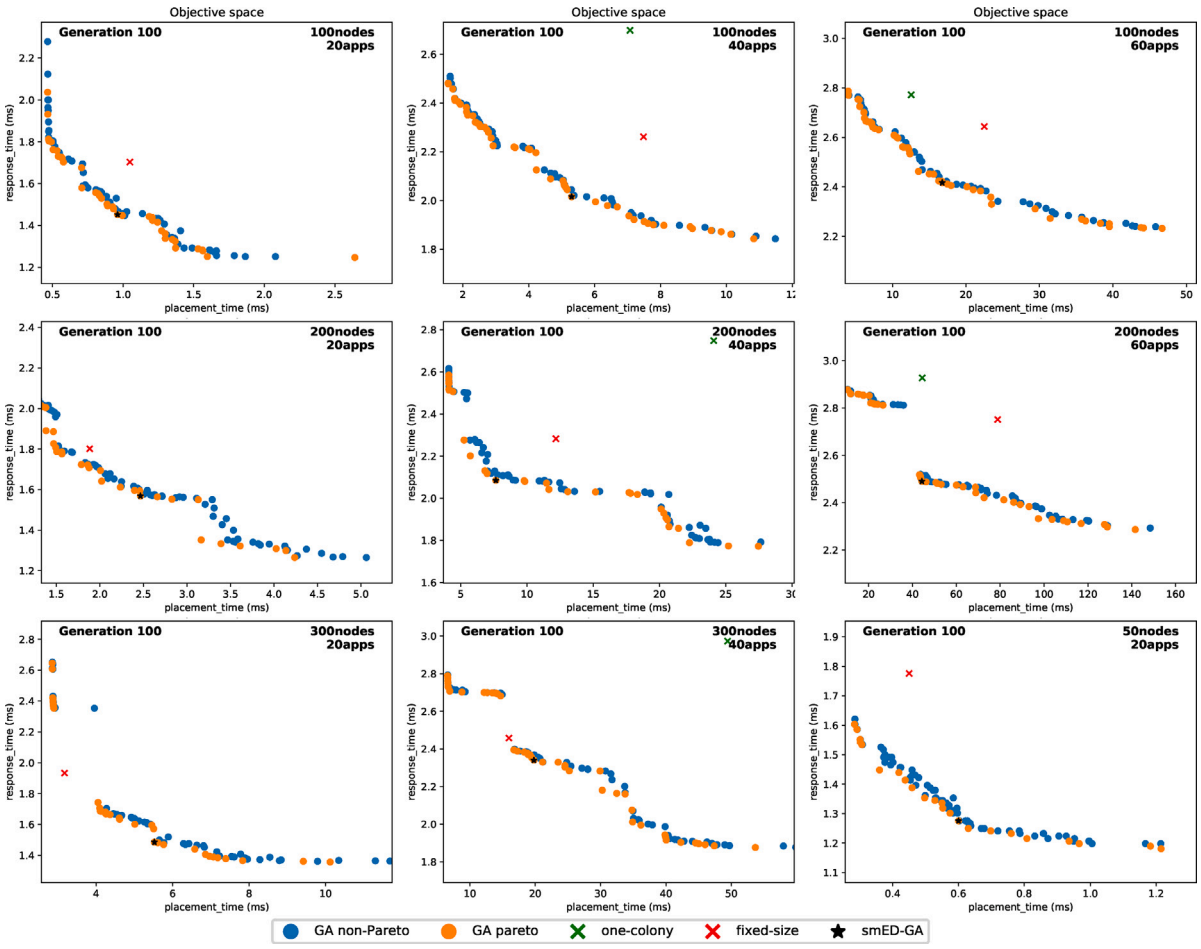


Fig. 8. Scatter plots that represent the objective space of the experiments.

the control algorithms is minimal. However, the trend indicates that this value increases as the complexity of the infrastructure increases. For instance, the experiments with 200 and 300 devices require a bigger number of generations, and experiments with 50 and 100 devices the smallest ones.

Additionally, we use the quality unary indicator $S(GA)$ (Zitzler & Thiele, 1999) to measure the distribution and range of the solutions in the Pareto front. This indicator represents the size of the objective space covered by the non-dominated solutions. As we are measuring two objectives, we calculate the area of the square defined by the left-top and right-bottom solutions in the Pareto front, using the normalized objective values of these two points. A bigger objective space volume indicates a better Pareto front. In our case, the biggest square area is 1.0 because we normalize the objective values. The values of $S(GA)$ in Table 3 indicate that the area of the Pareto front ranges from 0.627 to 0.904 for all the experiments. It is worth noting that an area of 1.0 is very unlikely, as it can only be obtained with an ideal Pareto front. However, values greater than 0.6 are a good indicator of the quality of the Pareto front (Singh, 2020).

To summarize, the quality of the Pareto front depends on the distribution of the solution across the objective space and the maximization of the range of each objective (Knowles & Corne, 2002). Having a set of solutions, instead of a single solution as in the case of the control algorithms, is an inherent advantage of multi-objective optimization techniques that are based on dominance and that result in a Pareto set. The Pareto set offers a wider number of alternatives to apply specific criteria for the selection of the solution that better fits each specific case.

Finally, it is also interesting to analyze the evolution of the GA's objective space on its own. A complete analysis of the evolution of the Pareto front requires comparing it across the generations of the GA. Due to the high number of plots required to show all the generations of all the experiment scenarios and all the execution repetitions ($100 \times 9 \times 10$), we have selected eight intermediate generations of one of the experiment executions of one scenario. Fig. 9 shows the objective space for generations 1, 5, 10, 20, 30, 40, 60, and 100 of the experiment with 200 nodes and 40 apps.

The number of solutions in the Pareto front increases (number of orange points) as the generations evolve. Additionally, these solutions become more homogeneously distributed along the Pareto front as the optimization progresses. As we explained before, optimization is better when both the number of solutions in the Pareto front and their homogeneous distribution increase, as it offers a wider range of solutions for the system administrator to choose a specific one by applying their criteria. It is also observed that the Pareto front evolves by approaching to the origin point of the objective space, i.e. by reducing the values of the optimization objectives.

About the improvement of the optimization values, we have also represented the normalized Euclidean distance of the closest solution to the origin point (*smallED-GA*) along the generations in Fig. 10. It is worth remembering that the *smallED-GA* is the closest solution to the origin point in the Pareto front that corresponds to the one with the most balanced improvement of the two optimization objectives. However, its evolution is a good indicator of the evolution of the Pareto front in terms of reducing the values of the objective.

Fig. 10 represents both the evolution of the *smallED-GA* solution for each single experiment execution (in pastel colors) and the evolution of

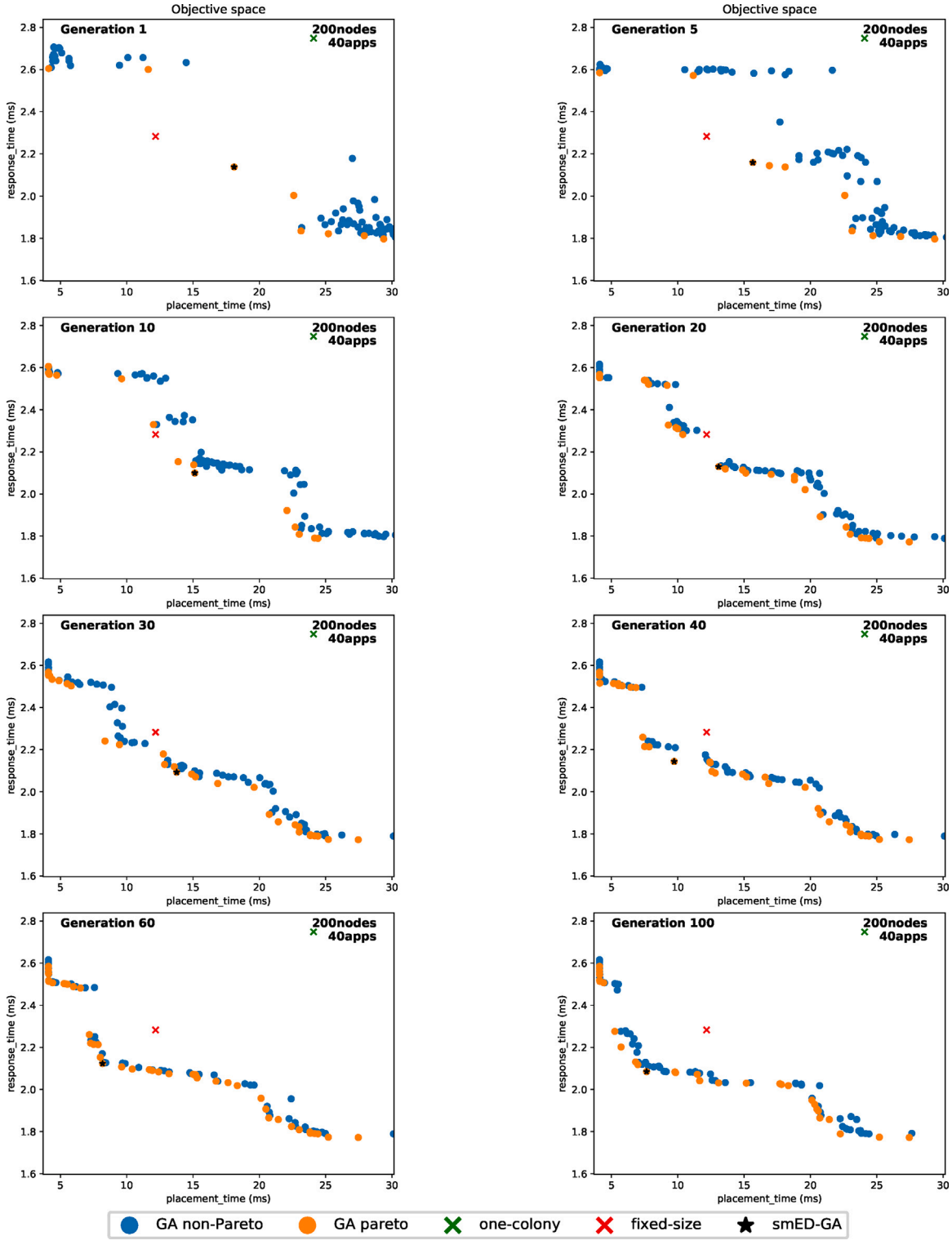


Fig. 9. Objective space evolution for the experiment with 200 nodes and 40 applications.

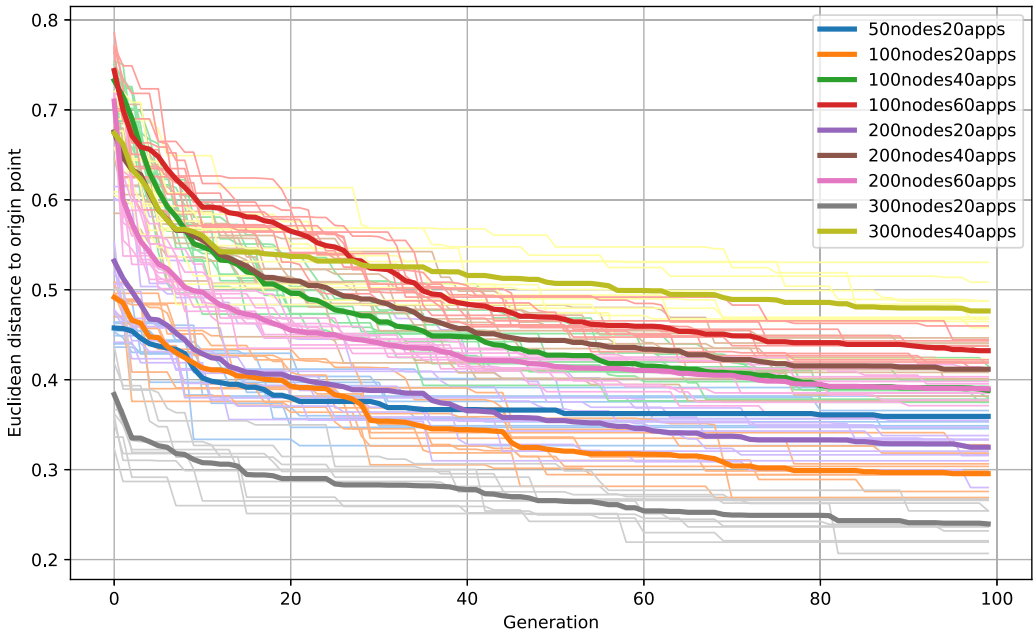


Fig. 10. Evolution of the solution with the smallest Euclidean distance to the origin point (*smallED-GA*) along the generations of the GA. Pastel color lines correspond to a single experiment repetition. Intensive color lines correspond to the mean values of the repetitions of each experiment scenario.

the average values of the repetitions grouped by experiment scenario (intensive colors). In all the cases, it is observed that the objective values are improved very fast in the first 20 generations. This reduction is decelerated for the period 20–80 generations. And, finally, the solutions almost reached a steady state from generation 80. This helps us to conclude that objectives are improved along generations and that 100 generations is a good ending condition for our experiment scenarios.

In summary, it is concluded that the quality of the Pareto front increases along the generations in terms of the range of objective values, distribution, and number of solutions. It is also concluded that the quality increase between generations is higher in the first generations than in the last ones, although there are still small improvements between generations in those last generations. Without loss of generality, the same trend is observed in all the executions of all the experiment scenarios.

7. Conclusion

Our work proposed improving current state-of-the-art optimization processes with a coordinated optimization of the fog colony layout and the service placement using a hybrid genetic algorithm, which integrates a greedy algorithm. The objective was to find a trade-off between complexity reduction of the optimization of the fog infrastructure using smaller-scale elements such as fog colonies, but without losing solution quality of applying local optimizations to the colonies instead of a global optimization of the whole fog infrastructure.

Our initial hypothesis was supported because the results have shown that our proposal obtains better quality results for the Pareto front and the closest solution to the origin point than the two control algorithms. One control algorithm defined the colony layout with a fixed size, while the other only considered one colony with all devices. Both placed the services in a predefined fog layout without the opportunity to adapt this layout.

The experiments have also shown the influence of the fog colony layout on the system performance metrics, the suitability of using a dendrogram to define the fog colony layout, and the effectiveness of a hybrid GA-based to outperform non-coordinated optimizations.

The optimization was based on the improvement of the execution time of the placement algorithm and the response time obtained by the users with the defined placement of the fog services. The results quality was evaluated for minimization of the values of the objectives, distribution in the solution space, and evolution of the solutions along the generations of the GA.

Once our study has supported the idea that a combined optimization of colony layout and service placement improves the quality of the solutions, future work should be primarily focused on improving the optimization process. For example, it is important to study if other new variants of GAs, or even different meta-heuristics, can be hybridized and if they would obtain additional improvement to the use of the NSGA-II. Additionally, it is necessary to investigate whether combining the GA with other meta-heuristics different from a greedy algorithm will enhance hybrid optimization.

Another future work considers incorporating additional optimization algorithms. For example, our proposal does not need to deal with overloaded colonies because services are shifted to neighbor colonies when required. However, if the infrastructure operator is concerned with a homogeneous distribution of the services along the colonies, the GA algorithm could incorporate some performance indicators of this homogeneous distribution as an additional optimization objective.

Funding

This work was supported by MICIU/AEI/10.13039/501100011033, Spain [grant number PID2021-128071OB-I00] and FEDER,UE, Spain [grant number PID2021-128071OB-I00].

CRediT authorship contribution statement

Francisco Talavera: Software, Validation, Visualization, Investigation, Writing – review & editing. **Isaac Lera:** Conceptualization, Data curation, Formal analysis, Funding acquisition, Investigation, Methodology, Project administration, Resources, Validation, Visualization, Writing – review & editing. **Carlos Juiz:** Conceptualization, Formal analysis, Funding acquisition, Resources, Writing – review & editing. **Carlos Guerrero:** Conceptualization, Data curation, Formal analysis, Funding acquisition, Investigation, Methodology, Project administration, Resources, Supervision, Validation, Visualization, Writing – original draft, Writing – review & editing.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

We have attached the link to the data/code

Source code of the genetic algorithm and data of the execution (Original data) (GitHub)
Pre-print version (TeX source) (arXiv)
.

References

Ayoubi, M., Ramezanpour, M., & Khorsand, R. (2021). An autonomous IoT service placement methodology in fog computing. *Software - Practice and Experience*, *51*(5), 1097–1120.

Azimzadeh, M., Rezaee, A., Jassbi, S. J., & Esnaashari, M. (2022). Placement of IoT services in fog environment based on complex network features: A genetic-based approach. *Cluster Computing*, *25*(5), 3423–3445.

Azizi, S., & Khosroabadi, F. (2019). A QoS-aware service placement algorithm for fog-cloud computing environments. In *4th international conference neural sciences (ICNS). mathematics and computer science*.

Azizi, S., Khosroabadi, F., & Shojafar, M. (2019). A priority-based service placement policy for fog-cloud computing systems. *Computational Methods for Differential Equations*, *7*(Issue 4 (Special Issue)), 521–534.

Birattari, M., & Kacprzyk, J. (2009). *Tuning metaheuristics: a machine learning perspective*: *vol. 197*, Springer.

Bonomi, F., Milito, R., Zhu, J., & Addepalli, S. (2012). Fog computing and its role in the Internet of Things. In *Proceedings of the first edition of the MCC workshop on mobile cloud computing* (pp. 13–16).

Brogi, A., Forti, S., Guerrero, C., & Lera, I. (2020). How to place your apps in the fog: State of the art and open challenges. *Software - Practice and Experience*, *50*(5), 719–740.

Buijs, J. C., van Dongen, B. F., & van der Aalst, W. M. (2012). A genetic algorithm for discovering process trees. In *2012 IEEE congress on evolutionary computation* (pp. 1–8). IEEE.

Deb, K., Pratap, A., Agarwal, S., & Meyarivan, T. (2002). A fast and elitist multiobjective genetic algorithm: NSGA-II. *IEEE Transactions on Evolutionary Computation*, *6*(2), 182–197.

Eslami, M., & Sakhaei-nia, M. (2021). A novel service deployment policy in fog computing considering the degree of availability and fog landscape utilization using multiobjective evolutionary algorithms. In *2021 12th international conference on information and knowledge technology* (pp. 163–168).

Ferreira, J. C., Fonseca, C. M., & Gaspar-Cunha, A. (2007). Methodology to select solutions from the Pareto-optimal set: A comparative study. In *Proceedings of the 9th annual conference on genetic and evolutionary computation* (pp. 789–796). New York, NY, USA: Association for Computing Machinery.

Ghaferi, E., Malekhosseini, R., Rad, F., & Bagherifard, K. (2022). A clustering method for locating services based on fog computing for the Internet of Things. *Journal of Supercomputing*, *78*(11), 13756–13779.

Ghobaei-Arani, M., Asadianfam, S., & Abolfathi, A. (2022). Deploying IoT services on the fog infrastructure: A graph partitioning-based approach. *Software - Practice and Experience*, *52*(9), 1967–1986.

Gibbs, M. S., Maier, H. R., & Dandy, G. C. (2015). Using characteristics of the optimisation problem to determine the genetic algorithm population size when the number of evaluations is limited. *Environmental Modelling and Software*, *69*, 226–239.

Guerrero, C., Lera, I., & Juiz, C. (2018). On the influence of fog colonies partitioning in fog application makespan. In *2018 IEEE 6th international conference on future internet of things and cloud* (pp. 377–384). IEEE.

Guerrero, C., Lera, I., & Juiz, C. (2020). Optimization policy for file replica placement in fog domains. *Concurrency Computations: Practice and Experience*, *32*(21), Article e5343.

Guerrero, C., Lera, I., & Juiz, C. (2022). Genetic-based optimization in fog computing: Current trends and research opportunities. *Swarm and Evolutionary Computation*, *72*, Article 101094.

Gupta, H., Vahid Dastjerdi, A., Ghosh, S. K., & Buyya, R. (2017). IFogSim: A toolkit for modeling and simulation of resource management techniques in the Internet of Things, edge and fog computing environments. *Software - Practice and Experience*, *47*(9), 1275–1296.

Hatti, D. I., & Sutagundar, A. V. (2021). Chapter 4 - swarm intelligence based MSMOPSO for optimization of resource provisioning in internet of things. In S. Bhattacharyya, P. Dutta, D. Samanta, A. Mukherjee, & I. Pan (Eds.), *Recent trends in computational intelligence enabled research* (pp. 61–82). Academic Press.

Holland, J. H., et al. (1992). *Adaptation in natural and artificial systems: an introductory analysis with applications to biology, control, and artificial intelligence*. MIT Press.

Hu, C., Shu, X., Yan, X., Zeng, D., Gong, W., & Wang, L. (2020). Inline wireless mobile sensors and fog nodes placement for leakage detection in water distribution systems. *Software - Practice and Experience*, *50*(7), 1152–1167.

Jalali Khalil Abadi, Z., & Mansouri, N. (2024). A comprehensive survey on scheduling algorithms using fuzzy systems in distributed environments. *Artificial Intelligence Review*, *57*(1), 4.

Knowles, J., & Corne, D. (2002). On metrics for comparing nondominated sets. In *Proceedings of the 2002 congress on evolutionary computation. cEC'02 (cat. no. 02TH8600)*: *1*, (pp. 711–716). IEEE.

Larranaga, P., Kuijpers, C. M. H., Murga, R. H., Inza, I., & Dizdarevic, S. (1999). Genetic algorithms for the Travelling Salesman Problem: A review of representations and operators. *Artificial Intelligence Review*, *13*(2), 129–170.

Lera, I., Guerrero, C., & Juiz, C. (2018). Availability-aware service placement policy in fog computing based on graph partitions. *IEEE Internet of Things Journal*, *6*(2), 3641–3651.

Liu, C., Wang, J., Zhou, L., & Rezaeipanah, A. (2022). Solving the multi-objective problem of IoT service placement in Fog computing using Cuckoo search algorithm. *Neural Processing Letters*, *54*(3), 1823–1854.

Lordan, F., Lezzi, D., & Badia, R. M. (2021). Colony: Parallel functions as a service on the cloud-edge continuum. In L. Sousa, N. Roma, & P. Tomás (Eds.), *Euro-par 2021: parallel processing* (pp. 269–284). Cham: Springer International Publishing.

Manihar, S., Patel, R., & Agrawal, S. (2024). A survey on mission critical task placement and resource utilization methods in the IoT fog-cloud environment. *Recent Trends in Computational Sciences*, 284–290.

Mann, Z. A. (2022). Decentralized application placement in fog computing. *IEEE Transactions on Parallel and Distributed Systems*, *33*(12), 3262–3273.

Minh, Q. T., Huu, P. N., Tsuchiya, T., & Toulouse, M. (2019). Openness in fog computing for the Internet of Things. In T. K. Dang, J. Küng, M. Takizawa, & S. H. Bui (Eds.), *Future Data and Security Engineering* (pp. 343–357). Cham: Springer International Publishing.

Minh, Q. T., Nguyen, D. T., Van Le, A., Nguyen, H. D., & Truong, A. (2017). Toward service placement on fog computing landscape. In *2017 4th NAFOSTeD conference on information and computer science* (pp. 291–296).

Mosayebi, M., & Sodhi, M. (2020). Tuning genetic algorithm parameters using design of experiments. In *Proceedings of the 2020 genetic and evolutionary computation conference companion* (pp. 1937–1944). New York, NY, USA: Association for Computing Machinery.

Moysiadis, V., Sarigiannidis, P., & Moscholios, I. (2018). Towards distributed data management in fog computing. *Wireless Communications and Mobile Computing*, *2018*, 14.

Murtagh, F., & Contreras, P. (2012). Algorithms for hierarchical clustering: An overview. *Wiley Interdisciplinary Reviews: Data Mining and Knowledge Discovery*, *2*(1), 86–97.

Nashaat, H., Ahmed, E., & Rizk, R. (2020). IoT application placement algorithm based on multi-dimensional QoE prioritization model in fog computing environment. *IEEE Access*, *8*, 111253–111264.

Natesha, B., & Guddeti, R. M. R. (2021). Adopting elitism-based genetic algorithm for minimizing multi-objective problems of IoT service placement in fog computing environment. *Journal of Network and Computer Applications*, *178*, Article 102972.

Nayeri, Z. M., Ghafarian, T., & Javadi, B. (2021). Application placement in fog computing with AI approach: Taxonomy and a state of the art survey. *Journal of Network and Computer Applications*, *185*, Article 103078.

Nguyen, Q.-H., & Truong Pham, T.-A. (2018). Studying and developing a resource allocation algorithm in fog computing. In *2018 international conference on advanced computing and applications* (pp. 76–82).

Nicopolitidis, P., Pham-Nguyen, H.-N., & Tran-Minh, Q. (2019). Dynamic resource provisioning on fog landscapes. *Security and Communication Networks*, *2019*, Article 1798391.

Nikolopoulos, V., Nikolaidou, M., Voreakou, M., & Anagnostopoulos, D. (2022a). Context diffusion in fog colonies: Exploring autonomous fog node operation using ECTORAS. *IoT*, *3*(1), 91–108.

Nikolopoulos, V., Nikolaidou, M., Voreakou, M., & Anagnostopoulos, D. (2022b). Fog node self-control middleware: Enhancing context awareness towards autonomous decision making in fog colonies. *Internet Things*, *19*, Article 100549.

Ogundoyin, S. O., & Kamil, I. A. (2022). Secure and privacy-preserving D2D communication in fog computing services. *Computer Networks*, *210*, Article 108942.

Rakshith, G., Rahul, M. V., Sanjay, G. S., Natesha, B. V., & Ram Mohana Reddy, G. (2018). Resource provisioning framework for IoT applications in fog computing environment. In *2018 IEEE international conference on advanced networks and telecommunications systems* (pp. 1–6).

Rodriguez, F. J., Garcia-Martinez, C., & Lozano, M. (2012). Hybrid metaheuristics based on evolutionary algorithms and simulated annealing: Taxonomy, comparison, and synergy test. *IEEE Transactions on Evolutionary Computation*, *16*(6), 787–800.

Salimian, M., Ghobaei-Arani, M., & Shahidinejad, A. (2021). Toward an autonomic approach for Internet of Things service placement using gray wolf optimization in the fog computing environment. *Software - Practice and Experience*, *51*(8), 1745–1772.

Salimian, M., Ghobaei-Arani, M., & Shahidinejad, A. (2022). An evolutionary multi-objective optimization technique to deploy the IoT services in fog-enabled networks: An autonomous approach. *Applied Artificial Intelligence*, *36*(1), Article 2008149.

Singh, H. K. (2020). Understanding hypervolume behavior theoretically for benchmarking in evolutionary multi/many-objective optimization. *IEEE Transactions on Evolutionary Computation*, *24*(3), 603–610.

Skarlat, O., Karagiannis, V., Rausch, T., Bachmann, K., & Schulte, S. (2018). A framework for optimization, service placement, and runtime operation in the fog. In A. Sill, & J. Spillner (Eds.), *11th IEEE/ACM international conference on utility and cloud computing* (pp. 164–173). IEEE Computer Society.

Skarlat, O., Nardelli, M., Schulte, S., Borkowski, M., & Leitner, P. (2017). Optimized IoT service placement in the fog. *Service Oriented Computing and Applications*, *11*(4), 427–443.

Skarlat, O., & Schulte, S. (2021). FogFrame: A framework for IoT application execution in the fog. *PeerJ Computer Science*, *7*, Article e588.

Srinivas, N., & Deb, K. (1994). Muiltiobjective optimization using nondominated sorting in genetic algorithms. *Evolutionary Computation*, *2*(3), 221–248.

Suarez, J. N., & Salcedo, A. (2017). ID3 and k-means based methodology for internet of things device classification. In *2017 international conference on mechatronics, electronics and automotive engineering* (pp. 129–133).

Talbi, E. G. (2002). A taxonomy of hybrid metaheuristics. *Journal of Heuristics*, *8*(5), 541–564.

Tavousi, F., Azizi, S., & Ghaderzadeh, A. (2022). A fuzzy approach for optimal placement of IoT applications in fog-cloud computing. *Cluster Computing*, *25*(1), 303–320.

Tran, M.-Q., Nguyen, D. T., Le, V. A., Nguyen, D. H., & Pham, T. V. (2019). Task placement on fog computing made efficient for IoT application provision. *Wireless Communications and Mobile Computing*, *2019*, Article 6215454.

Tran, Q. M., Nguyen, P. H., Tsuchiya, T., & Toulouse, M. (2020). Designed features for improving openness, scalability and programmability in the fog computing-based IoT systems. *SN Computer Science*, *1*(4), 194.

Venticinque, S., & Amato, A. (2019). A methodology for deployment of IoT application in fog. *Journal of Ambient Intelligence and Humanized Computing*, *10*(5), 1955–1976.

Von Lücken, C., Barán, B., & Brizuela, C. (2014). A survey on multi-objective evolutionary algorithms for many-objective problems. *Computational Optimization and Applications*, *58*, 707–756.

Wang, N., & Varghese, B. (2022). Context-aware distribution of fog applications using deep reinforcement learning. *Journal of Network and Computer Applications*, *203*, Article 103354.

Zitzler, E., Laumanns, M., & Thiele, L. (2001). SPEA2: Improving the strength Pareto evolutionary algorithm. *TIK-Report*, *103*.

Zitzler, E., & Thiele, L. (1999). Multiobjective evolutionary algorithms: A comparative case study and the strength Pareto approach. *IEEE Transactions on Evolutionary Computation*, *3*(4), 257–271.